
EMPLOYMENT MANAGEMENT SYSTEM

Employees

The employee directory, endpoint by endpoint and flow by flow

EMPLOYEE MANAGEMENT FLOWS • 2 OCTOBER 2026

Contents

01	1. The files	3
02	2. The security decision	3
03	3. DTOs and why there are three of them	4
04	4. The mapper	5
05	5. Flow A - Browse the employee list (paginated)	6
06	6. Flow B - Search by name	7
07	7. Flow C - Create an employee	8
08	8. Flow D - Edit an employee	10
09	9. Flow E - Delete an employee	11
10	10. <code>GET /api/employees</code> and <code>GET /api/employees/{id}</code>	12
11	11. Employee endpoint summary	12

The employee directory. **Every endpoint in this feature is manager-level only** (`ADMIN` or `MANAGER`). An `EMPLOYEE` gets `403` on all seven, including reads.

1. The files

File	Role
<code>employee/EmployeeController.java</code>	7 endpoints, HTTP only
<code>employee/EmployeeService.java</code>	interface
<code>employee/EmployeeServiceImp.java</code>	the business rules
<code>employee/EmployeeRepository.java</code>	Spring Data queries
<code>employee/Employee.java</code>	the JPA entity
<code>employee/dto/EmployeeCreateRequest.java</code>	validated create body
<code>employee/dto/EmployeeUpdateRequest.java</code>	partial update body
<code>employee/dto/EmployeeResponse.java</code>	what the frontend receives
<code>employee/mapper/EmployeeMapper.java</code>	MapStruct conversions
Frontend	<code>pages/EmployeesPage.jsx</code> , <code>components/EmployeeFormModal.jsx</code> , <code>api/http.js</code>

2. The security decision

```

@RestController
@RequestMapping("/api/employees")
@RequiredArgsConstructor
// Security model: the whole employee directory is ADMIN/MANAGER only, reads included.
// An employee must never enumerate other staff or read their salary.
@PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
public class EmployeeController {

```

The annotation sits on the **class**, so it applies to all seven methods with no repetition. Two reasons reads are included, not just writes:

- Salary is sensitive.** `EmployeeResponse` contains `salary` . If employees could read the directory they would see everyone's pay.

2. Enumeration is sensitive. Even without salary, a list of all staff is information an employee should not have by default.

There is deliberately **no** "get my own employee record" endpoint here. The only self-service the app offers is attendance. If you add one later, it must be a *separate* method annotated `hasRole('EMPLOYEE')` (or `hasAnyRole('EMPLOYEE', 'ADMIN', 'MANAGER')`) - do not remove the class-level annotation, or the directory reopens.

`@PreAuthorize` runs in the AOP proxy around the controller method, i.e. after the JWT filter authenticated you but before the method body. `AccessDeniedException` is converted to `403 Forbidden: insufficient permissions` by `GlobalExceptionHandler:79`.

3. DTOs and why there are three of them

```
EmployeeCreateRequest → Employee (entity) → EmployeeResponse
EmployeeUpdateRequest → Employee (entity) → EmployeeResponse
```

TEXT

- **Request DTOs** are what the client may send. They are validated, and they are the *only* thing Jackson binds to, so a client cannot smuggle in `id` or a `department` object.
- **The entity** is the database shape.
- **The response DTO** is the safe shape. It exposes `department` as a nested `DepartmentSummary { id, name }` instead of the whole `Department` entity - which would drag along `employeeList` and recurse back into every employee.

`EmployeeResponse` (lines 13-32) contains: `id`, `firstName`, `lastName`, `email`, `phone`, `hireDate`, `salary`, `department {id, name}`.

Create validation (EmployeeCreateRequest)

Field	Rule	Error message
<code>firstName</code>	<code>@NotBlank</code>	First name is required
<code>lastName</code>	<code>@NotBlank</code>	Last name is required
<code>email</code>	<code>@NotBlank</code> + <code>@Email</code>	Email is required / invalid email format
<code>phone</code>	<code>@NotBlank</code>	Phone number is required
<code>hireDate</code>	none (optional)	-
<code>salary</code>	<code>@DecimalMin(value="0.0", inclusive=false)</code>	Salary must be positive
<code>departmentName</code>	<code>@NotBlank</code>	Department name is required

`inclusive = false` means the value must be **greater than** 0, so `0` is rejected too. Note `hireDate` has no validation at all - a garbage date string becomes a `400` from Jackson's date parsing (`Malformed request body`).

Update validation (EmployeeUpdateRequest)

Every field is **optional** - no `@NotBlank`. Only `@Email` and `@DecimalMin` remain, and even those are skipped when the value is `null`. This is what makes it a PATCH: absent fields mean "leave unchanged".

Two things to notice:

- `hireDate` is missing from `EmployeeUpdateRequest`. That is why `EmployeeFormModal` only renders the hire-date input when `mode ≡ "create"` (`EmployeeFormModal.jsx:118`). It is a known gap, not an oversight you can fix in the UI - the field has to be added to the DTO first.
- There is **no way to null a field** (you cannot clear a phone number), because a `null` is indistinguishable from "not supplied". That is the standard trade-off of PATCH.

4. The mapper

```

@Mapper(componentModel = "spring",
        nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IGNORE)
public interface EmployeeMapper {

    @Mapping(target = "department", ignore = true)
    Employee toEntity(EmployeeCreateRequest req);

    @Mapping(target = "department", ignore = true)
    void updateEmployee(EmployeeUpdateRequest updateReq, @MappingTarget Employee emp);

    EmployeeResponse toDtoResponse(Employee emp);
}

```

Three things to explain:

1. `@Mapping(target = "department", ignore = true)` - there is **no** `departmentName` field on `Employee` (the entity has a `Department department` object). Without `ignore`, MapStruct would try to map `departmentName` -> `department` and fail to compile. The service sets the relationship explicitly instead.
2. `@MappingTarget Employee emp` - the second parameter is the *existing object to modify*, not a new one. MapStruct copies the non-null fields onto it. The method returns `void` because it mutates in place. This is what makes the update a JPA **dirty check** rather than a delete+insert.
3. `NullValuePropertyMappingStrategy.IGNORE` - **this is the setting that implements PATCH semantics**. Without it, a `null` in the request would overwrite the field with `null`. With it, nulls are skipped.

`toDtoResponse` needs no annotations: every `EmployeeResponse` field has a same-named counterpart on `Employee`, except `department`, which is mapped from `Employee.department` into the nested `DepartmentSummary` (only `id` and `name` are matched).

`EmployeeMapper` is generated at compile time into `target/generated-sources/.../EmployeeMapperImpl.java`. If you change a DTO you must run a clean build.

5. Flow A - Browse the employee list (paginated)

Endpoint: `GET /api/employees/paged?page=0&size=10`

Screen: `EmployeesPage.jsx:20-25`

```
const { data, isLoading, isError, error } = useQuery({
  queryKey: ["employees", page, search],
  queryFn: () => search.trim() ? searchEmployee(search.trim()) : fetchEmployees(page),
  enabled: canManage,
});
```

- The query key contains `page` and `search`, so changing either triggers a different cache entry.
- `enabled: canManage` means the request is **never sent** for an employee. The page would redirect anyway, but this prevents a guaranteed-`403` request from firing during the brief render before the redirect.
- `search.trim() ? ... : ...` chooses the endpoint: searching switches from the paginated list to the flat search list.

The `queryFn` picks one of two response shapes, and the render code copes with both:

```
const rows = search.trim() ? (data ?? []) : (data?.content ?? []);
```

- `paged` -> `Page`, so rows are in `data.content`
- `search` -> `List`, so rows are the array itself

Backend:

`EmployeeController.getEmployeesPaged(Pageable pageable)` (line 48) takes the `Pageable` as a parameter with no annotation. Spring Data has a built-in resolver: any `page`, `size` and `sort` query parameters are bound automatically.

```
return ResponseEntity.ok(employeeService.getAllEmployees(pageable));
```

```
@Transactional
public Page<EmployeeResponse> getAllEmployees(Pageable pageable) {
    return employeeRepository.findAll(pageable).map(employeeMapper::toDtoResponse);
}
```

```
}
```

The comment `// page has already built-in map func.` means: `Page` is both the request-side container and the result, so `.map(...)` converts elements lazily without building a list.

One subtlety worth explaining: `getAllEmployees(Pageable)` is the only read method here without an explicit `@Transactional(readOnly = true)` annotation. It still works because the **class-level** `@Transactional` on `EmployeeServiceImp` covers it: the whole method body - including the `.map(...)` call, which is what touches the lazy `department` - executes inside that transaction. The other read methods annotate themselves `readOnly = true` as an optimisation and as documentation. (Contrast `AttendanceServiceImp.searchAttendance`, where the `readOnly` transaction is *load-bearing* because there is no class-level annotation - see `05-attendance.md` section 6.)

Frontend pagination UI: `page` is 0-based; the UI displays `page + 1`. "Next" is disabled when `page + 1 ≥ totalPages`.

6. Flow B - Search by name

Endpoint: `GET /api/employees/search?name=jane`

Frontend: `searchEmployee(name)` in `http.js:38`. Note there is **no** `encodeURIComponent` - a name containing `&` or a space would break the query string. Names with spaces work because the browser encodes them, but this is a latent bug.

The UI keeps two pieces of state on purpose:

```
const [searchInput, setSearchInput] = useState(""); // what's in the box
const [search, setSearch]           = useState(""); // what's been submitted
function handleSubmit(e) { e.preventDefault(); setSearch(searchInput); setPage(0); }
```

`searchInput` drives the input; `search` drives the query key. So typing does **not** fire a request per keystroke - only pressing "Search" (submitting the form) does.

Backend:

```
@Query("SELECT DISTINCT e FROM Employee e " +
      "LEFT JOIN FETCH e.department d " +
      "WHERE LOWER(CONCAT(e.firstName, ' ', e.lastName)) LIKE LOWER(CONCAT('%', :fullName, '%'))")
List<Employee> findByFullNameContainingWithDepartment(@Param("fullName") String fullName);
```

Explaining this query line by line:

- `CONCAT(e.firstName, ' ', e.lastName)` - concatenates into `"Jane Smith"` so one LIKE can match across the name boundary.

- `LOWER( ... ) LIKE LOWER( ... )` - case-insensitive matching. Note the `LOWER` is applied in Java, **not** in the SQL function name, so it works identically on MySQL and any other database.
- `'%' + :fullName + '%'` - contains, not starts-with.
- `LEFT JOIN FETCH e.department d` - fetches the department in the same query. This is **required**: without it, mapping to `EmployeeResponse` would touch the lazy `department` outside a safe context and throw `LazyInitializationException`.
- `DISTINCT` - needed because the join could otherwise duplicate a row if an employee matched by several joined rows.
- `@Param("fullName")` - binds the method parameter to the named query token.

`getEmployeesByFullNameContaining` then maps:

```
return employees.stream().map(employeeMapper::toDtoResponse).collect(Collectors.toList());
```

JAVA

Known limitation: search results are **not paginated** and there is no `size` limit. With thousands of employees this returns everything that matches. That is why the frontend hides the Prev/Next controls when searching (`{!isSearching && ( ... )}`).

7. Flow C - Create an employee

Endpoint: `POST /api/employees` -> `201 Created`

Screen: `EmployeeFormModal.jsx` with `mode="create"`.

Backend, `createEmployee` (line 32), step by step:

```
if (employeeRepository.existsByEmailIgnoreCase(request.getEmail().trim())) {
    throw new DuplicateResourceException("Email already exists");
}
```

JAVA

1. **Duplicate email check** -> `409 Conflict`. This is a *friendly* pre-check; the database's `unique` constraint on `email` is the real guarantee, and if it ever fires you would get `409` from `DataIntegrityViolationException` instead. `.trim()` means `" a@b.com "` and `"a@b.com"` are considered the same.

```
Employee employee = employeeMapper.toEntity(request);
```

JAVA

2. Convert the DTO to an entity. `hireDate`, `salary`, names and email are copied; `department` is skipped (`ignore = true`).

```

Department department =
departmentRepository.findByNameIgnoreCase(request.getDepartmentName().trim())
    .orElseThrow(() -> new ResourceNotFoundException("Department Not Found: " +
request.getDepartmentName()));
employee.setDepartment(department);

```

JAVA

3. The department is referenced by **name**, not **id** in the API. It is resolved case-insensitively; an unknown name -> `404 Department Not Found: X`. Notice this is why `@NotBlank` on `departmentName` matters: without it a `null` name would NPE on `.trim()`.

```

Employee saved = employeeRepository.save(employee);
return employeeMapper.toDtoResponse(saved);

```

JAVA

4. `save` on a new entity with a null id issues an `INSERT`; the returned object has the generated id. The response is the created employee, so the frontend could use the id without refetching (it refetches anyway, via cache invalidation).

Front-end form wiring:

```

const [form, setForm] = useState({
  firstName: employee?.firstName || "", // "" when creating
  ...
  salary: employee?.salary ?? "", // ?? not || so a salary of 0 is preserved
});
function handleChange(e) {
  setForm((prev) => ({ ...prev, [e.target.name]: e.target.value }));
}

```

JSX

- `employee?.x || ""` - optional chaining plus default, so the same component serves create and edit.
- `employee?.salary ?? ""` - **nullish** coalescing, so `salary: 0` stays `0`. With `||` it would become `""`.
- `handleChange` uses `e.target.name`, so every input just needs a `name` attribute - no per-field handler.
- HTML validation: `required` on first/last/email/phone/department; `type="email"` and `min="0"` on salary. These are UX-level checks; the backend `@Valid` checks are the ones that actually protect the data.
- `min="0"` in HTML allows `0`, but the backend's `@DecimalMin(inclusive = false)` rejects it - a small frontend/backend mismatch.

Submission:

```

const { mutate, isPending, isMutationError } = useMutation({
  mutationFn: ({ id, employeeFormData }) =>
    mode === "create" ? createEmployee(employeeFormData) : updateEmployee(id, employeeFormData),
  onSuccess: () => {
    pushToast(mode === "create" ? "Employee Created" : "Employee Updated");
    queryClient.invalidateQueries({ queryKey: ["employees"] });
    onClose();
  }
});

```

JSX

```
},
  onError: (error) => pushToast(error.message, "error"),
});
```

- One mutation serves both create and update.
- `invalidateQueries({ queryKey: ["employees"] })` - the key is a **prefix**, so every cached page (`["employees", 0, ""]`, `["employees", 1, ""]`, ...) is marked stale and refetched. That is how the list refreshes without a manual `refetch`.
- `onClose()` unmounts the modal after success.
- Errors become red toasts carrying the backend `message`.

8. Flow D - Edit an employee

Endpoint: `PATCH /api/employees/{id}`

Screen: the same modal, `mode="edit"`, seeded from the clicked row.

`updateEmployee` (line 89) is the most interesting service method:

```
Employee employee = employeeRepository.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException("Employee not found"));
```

JAVA

1. Load the **managed** entity. Because the class is `@Transactional`, this entity is attached to the persistence context and Hibernate will detect changes at flush time. Nothing needs to be "saved" explicitly at the end.

```
if (request.getEmail() != null &&
    !employee.getEmail().equalsIgnoreCase(request.getEmail()) &&
    employeeRepository.existsByEmailIgnoreCase(request.getEmail())) {
    throw new DuplicateResourceException("Email already exists");
}
```

JAVA

2. The email uniqueness check has **three** conditions, and all of them matter:

- `getEmail() != null` - only check when an email was actually sent;
- `!employee.getEmail().equalsIgnoreCase(request.getEmail())` - **skip the check when the email did not change**, otherwise editing only the phone number would fail with "Email already exists" (the employee matches their own email);
- `existsByEmailIgnoreCase( ... )` - only fail if *someone else* has it.

3. Department, only if supplied:

```
Department department = null;
if (request.getDepartmentName() != null) {
    department = departmentRepository.findByNameIgnoreCase(request.getDepartmentName())
}
```

JAVA

```
        .orElseThrow(() -> new ResourceNotFoundException("Department not found"));
    }
}
```

The local variable starts as `null` so that "not supplied" and "supplied and found" can be distinguished afterwards.

```
employeeMapper.updateEmployee(request, employee);
if (department != null) employee.setDepartment(department);
return employeeMapper.toDtoResponse(employee);
```

JAVA

4. MapStruct copies the non-null fields (`IGNORE` strategy), then the relationship is set separately. There is **no** `employeeRepository.save(employee)` call - that is the whole point of dirty checking: the entity is already managed, so Hibernate issues `UPDATE`s for changed columns on commit. The response is the mutated entity mapped to a DTO, so the UI sees the new values.

PATCH vs PUT in this API: employees use `PATCH /api/employees/{id}` but departments use `PUT /api/departments/{id}`. In practice both behave as partial updates, because `EmployeeUpdateRequest` and `DepartmentUpdateReq` both ignore nulls. The methods differ, which is technically inconsistent REST.

9. Flow E - Delete an employee

Endpoint: `DELETE /api/employees/{id}` -> `204 No Content`

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteEmployee(@PathVariable Long id){ employeeService.deleteEmployee(id); }
```

JAVA

The controller returns `void` with `@ResponseStatus(NO_CONTENT)`, so the body is empty and `http.js:27` (`if (response.status === 204) return null;`) handles it.

```
Employee emp = employeeRepository.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException("Employee not found"));
employeeRepository.delete(emp);
```

JAVA

The row is loaded first for a better error message (`404` instead of a silent no-op). `delete` triggers:

- the `users.employee_id` FK - if a login account is linked, the DB will refuse (or set null, depending on the DDL Hibernate generated). If it refuses, you get `409` from `DataIntegrityViolationException`.
- `Attendance.attendanceList` is `cascade = ALL, orphanRemoval = true`, so the employee's attendance rows are deleted with them - that is why the employee delete is a heavy operation.

Frontend:

```
function handleDelete(id) {
  if (window.confirm("Delete this employee?")) deleteMutate(id);
}
```

JSX

`window.confirm` is a blocking browser dialog, then a mutation whose `onSuccess` toasts "Employee Deleted" and invalidates `["employees"]`.

There is no error toast for a failed delete at the page level beyond the mutation's `onError`, and the row is not optimistically removed - the refetch after invalidation is what makes it disappear.

10. GET `/api/employees` and GET `/api/employees/{id}`

Two endpoints the employee *page* does not use:

- GET `/api/employees` -> `List<EmployeeResponse>` (unpaged). Used by the **Dashboard** (to count employees and to populate the attendance employee dropdown) and by `AttendancePage` for the manager's dropdown.
- GET `/api/employees/{id}` -> one employee. Exposed but not called from the current frontend; it is manager-only like everything else.

Routing detail worth knowing: `@GetMapping("/{id}")` coexists with `@GetMapping("/paged")` and `@GetMapping("/search")`. Spring's path matching prefers the **literal** pattern over the `{id}` template, so `/api/employees/search` reaches the search method and never tries to bind `"search"` to a `Long`. If the literal mappings were removed, `/search` would produce a `400` from type conversion.

11. Employee endpoint summary

Method	Path	Body / Params	Success	Errors
POST	<code>/api/employees</code>	<code>EmployeeCreateRequest</code>	<code>201 EmployeeResponse</code>	400 validation, 404 unknown department, 409 email exists
GET	<code>/api/employees</code>	-	<code>200 [EmployeeResponse]</code>	-
GET	<code>/api/employees/{id}</code>	-	<code>200 EmployeeResponse</code>	404
GET	<code>/api/employees/paged</code>	<code>page</code> , <code>size</code> , <code>sort</code>	<code>200 Page<EmployeeResponse></code>	-

Method	Path	Body / Params	Success	Errors
GET	/api/employees/search	name	200 [EmployeeResponse]	400 if name missing
PATCH	/api/employees/{id}	EmployeeUpdateRequest	200 EmployeeResponse	400 validation, 404 employee/departement, 409 email exists
DELETE	/api/employees/{id}	-	204 empty	404, 409 FK conflict

All require `ROLE_ADMIN` or `ROLE_MANAGER` ; an `ROLE_EMPLOYEE` gets `403` on all seven.